



Software Engineering

(CSC 364)

Unit 2: Software Processes

Rajendra Kandel

Software Process

- A software process is a set of related activities that leads to the production of a software system.
- There are many different types of software systems, and there is no universal software engineering method that is applicable to all of them.
- Consequently, there is no universally applicable software process.
- The process used in different companies depends on the type of software being developed, the requirements of the software customer, and the skills of the people writing the software.

Software Process

The four fundamental software engineering activities

1. **Software specification** : The functionality of the software and constraints on its operation must be defined.
2. **Software development** : The software to meet the specification must be produced.
3. **Software validation** : The software must be validated to ensure that it does what the customer wants.
4. **Software evolution** : The software must evolve to meet changing customer needs.

Software Process

- These activities are complex activities in themselves, and they include sub activities such as requirements validation, architectural design, and unit testing.
- Processes also include other activities, such as software configuration management and project planning that support production activities.
- When we describe and discuss processes, we usually talk about the activities in these processes, such as specifying a data model and designing a user interface, and the ordering of these activities.
- We can all relate to what people do to develop software.

Software Process

- However, when describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities:
 - **Products or deliverables are the outcomes of a process activity.** For example, the outcome of the activity of architectural design may be a model of the software architecture.
 - **Roles reflect the responsibilities of the people involved in the process.** Examples of roles are project manager, configuration manager, and programmer.
 - **Pre- and postconditions are conditions that must hold before and after a process activity has been enacted or a product produced.** For example, before architectural design begins, a precondition may be that the consumer has approved all requirements; after this activity is finished, a postcondition might be that the UML models describing the architecture have been reviewed.

Software Process Models

- A software process model (sometimes called a Software Development Life Cycle or SDLC model) is a simplified representation of a software process.
- Each process model represents a process from a particular perspective and thus only provides partial information about that process.
- For example, a process activity model shows the activities and their sequence but may not show the roles of the people involved in these activities.
- This unit introduce a number of very general process models (sometimes called process paradigms) and present these from an architectural perspective. That is, we see the framework of the process but not the details of process activities.

Software Process Models

- **The waterfall model** : This takes the fundamental process activities of specification, development, validation, and evolution and represents them as separate process phases such as requirements specification, software design, implementation, and testing.
- **Incremental development** : This approach interleaves the activities of specification, development, and validation. The system is developed as a series of versions (increments), with each version adding functionality to the previous version.
- **Integration and configuration** : This approach relies on the availability of reusable components or systems. The system development process focuses on configuring these components for use in a new setting and integrating them into a system.

Waterfall Model

- The first published model of the software development process was derived from engineering process models used in large military systems engineering (Royce 1970).
- It presents the software development process as a number of stages.
- Because of the cascade from one phase to another, this model is known as the waterfall model or software life cycle.
- The waterfall model is an example of a plan-driven process.
- In principle at least, you plan and schedule all of the process activities before starting software development.

Waterfall Model

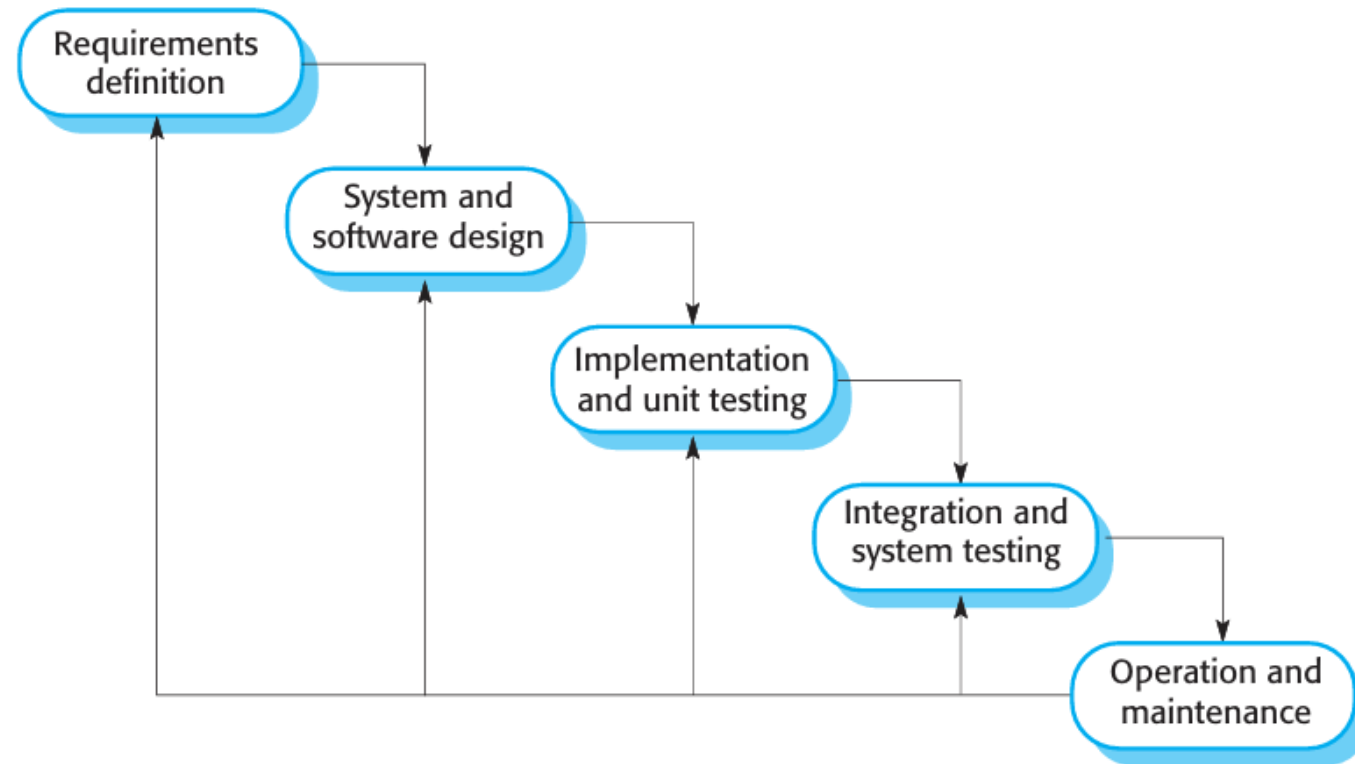


Figure 2.1 The waterfall model

Waterfall Model

The stages of the waterfall model directly reflect the fundamental software development activities:

- **Requirements analysis and definition** : The system's services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
- **System and software design** : The systems design process allocates the requirements to either hardware or software systems. It establishes an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.

Waterfall Model

- **Implementation and unit testing** : During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.
- **Integration and system testing** : The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
- **Operation and maintenance** : Normally, this is the longest life-cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors that were not discovered in earlier stages of the life cycle, improving the implementation of system units, and enhancing the system's services as new requirements are discovered.

Waterfall Model

- In principle, the result of each phase in the waterfall model is one or more documents that are approved (“signed off”).
- The following phase should not start until the previous phase has finished. For hardware development, where high manufacturing costs are involved, this makes sense.
- However, for software development, these stages overlap and feed information to each other.
- During design, problems with requirements are identified; during coding design problems are found, and so on.
- The software process, in practice, is never a simple linear model but involves feedback from one phase to another.

Incremental Development

- Incremental development is based on the idea of developing an initial implementation, getting feedback from users and others, and evolving the software through several versions until the required system has been developed.
- Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.
- Incremental development in some form is now the most common approach for the development of application systems and software products. This approach can be either plan-driven, agile or, more usually, a mixture of these approaches.

Incremental Development

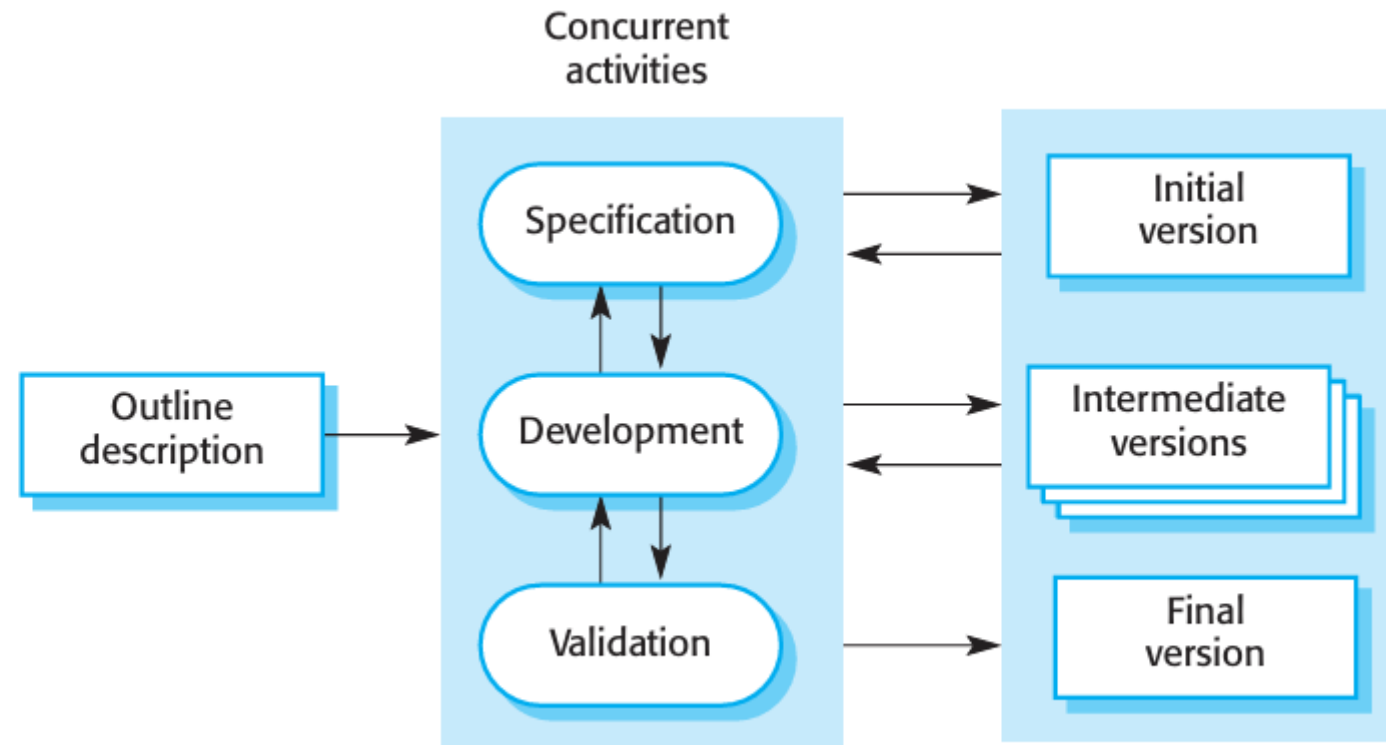


Figure 2.2 Incremental development

Incremental Development

- In a plan-driven approach, the system increments are identified in advance; if an agile approach is adopted, the early increments are identified, but the development of later increments depends on progress and customer priorities.
- Incremental software development, which is a fundamental part of agile development methods, is better than a waterfall approach for systems whose requirements are likely to change during the development process.
- This is the case for most business systems and software products. Incremental development reflects the way that we solve problems.
- By developing the software incrementally, it is cheaper and easier to make changes in the software as it is being developed.

Incremental Development

Incremental development has three major advantages over the waterfall model:

- The cost of implementing requirements changes is reduced. The amount of analysis and documentation that has to be redone is significantly less than is required with the waterfall model.
- It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented. Customers find it difficult to judge progress from software design documents.
- Early delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included. Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental Development

From a management perspective, the incremental approach has two problems:

- **The process is not visible.** Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost effective to produce documents that reflect every version of the system.
- **System structure tends to degrade as new increments are added.** Regular change leads to messy code as new functionality is added in whatever way is possible. It becomes increasingly difficult and costly to add new features to a system. To reduce structural degradation and general code messiness, agile methods suggest that you should regularly refactor (improve and restructure) the software.

Integration and Configuration

- In the majority of software projects, there is some software reuse. This often happens informally when people working on the project know of or search for code that is similar to what is required.
- They look for these, modify them as needed, and integrate them with the new code that they have developed.
- This informal reuse takes place regardless of the development process that is used. However, since 2000, software development processes that focus on the reuse of existing software have become widely used.
- Reuse-oriented approaches rely on a base of reusable software components and an integrating framework for the composition of these components.

Integration and Configuration

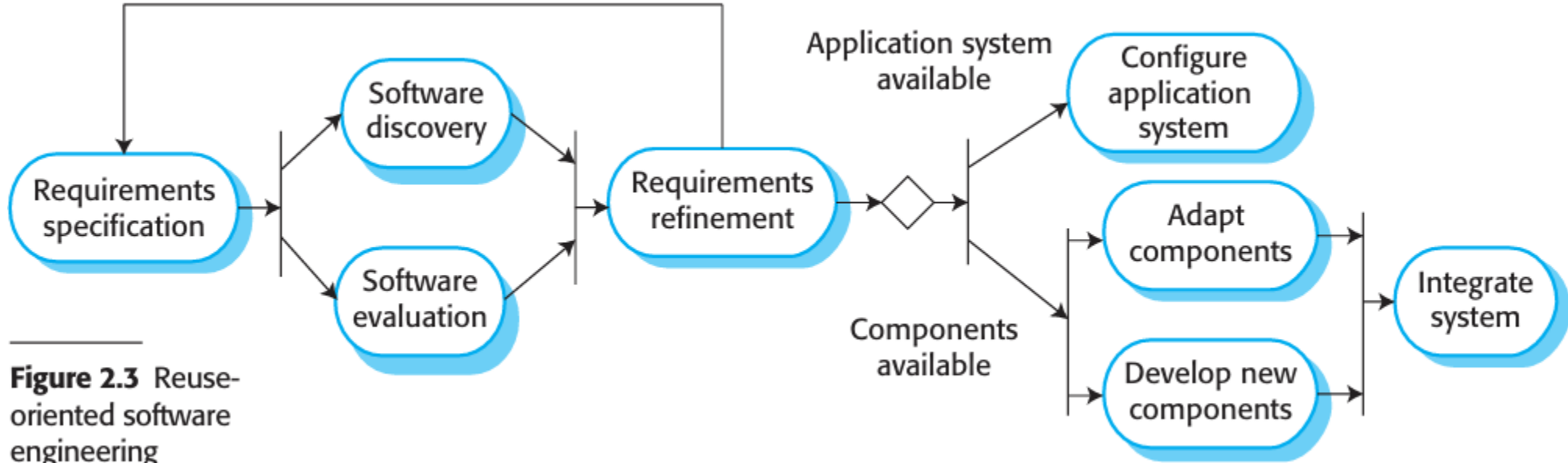


Figure 2.3 Reuse-oriented software engineering

Integration and Configuration

Figure shows a general process model for reuse-based development, based on integration and configuration. The stages in this process are:

- **Requirements specification** : The initial requirements for the system are proposed. These do not have to be elaborated in detail but should include brief descriptions of essential requirements and desirable system features.
- **Software discovery and evaluation** : Given an outline of the software requirements, a search is made for components and systems that provide the functionality required. Candidate components and systems are evaluated to see if they meet the essential requirements and if they are generally suitable for use in the system.

Integration and Configuration

- **Requirements refinement** : During this stage, the requirements are refined using information about the reusable components and applications that have been discovered. The requirements are modified to reflect the available components, and the system specification is re-defined. Where modifications are impossible, the component analysis activity may be reentered to search for alternative solutions.
- **Application system configuration** : If an off-the-shelf application system that meets the requirements is available, it may then be configured for use to create the new system.
- **Component adaptation and integration** : If there is no off-the-shelf system, individual reusable components may be modified and new components developed. These are then integrated to create the system.

Integration and Configuration

Three types of software components are frequently reused:

1. Stand-alone application systems that are configured for use in a particular environment. These systems are general-purpose systems that have many features, but they have to be adapted for use in a specific application.
2. Collections of objects that are developed as a component or as a package to be integrated with a component framework such as the Java Spring framework (Wheeler and White 2013).
3. Web services that are developed according to service standards and that are available for remote invocation over the Internet.

Integration and Configuration

- Reuse-oriented software engineering, based around configuration and integration, has the obvious advantage of reducing the amount of software to be developed and so reducing cost and risks.
- It usually also leads to faster delivery of the software. However, requirements compromises are inevitable, and this may lead to a system that does not meet the real needs of users.
- Furthermore, some control over the system evolution is lost as new versions of the reusable components are not under the control of the organization using them.

Software Process Activities

- Real software processes are interleaved sequences of technical, collaborative, and managerial activities with the overall goal of specifying, designing, implementing, and testing a software system. Generally, processes are now tool-supported.
- This means that software developers may use a range of software tools to help them, such as requirements management systems, design model editors, program editors, automated testing tools, and debuggers.
- The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes.
- In the water fall model, they are organized in sequence, whereas in incremental development they are interleaved.

Software Process Activities

- How these activities are carried out depends on the type of software being developed, the experience and competence of the developers, and the type of organization developing the software.
- The software process activities are:
 - **Software Specification**
 - **Software Design and Implementation**
 - **Software Validation**
 - **Software Evolution**

Software Specification

- Software specification or requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development.
- Requirements engineering is a particularly critical stage of the software process, as mistakes made at this stage inevitably lead to later problems in the system design and implementation.
- Before the requirements engineering process starts, a company may carry out a feasibility or marketing study to assess whether or not there is a need or a market for the software and whether or not it is technically and financially realistic to develop the software required.
- Feasibility studies are short-term, relatively cheap studies that inform the decision of whether or not to go ahead with a more detailed analysis.

Software Specification

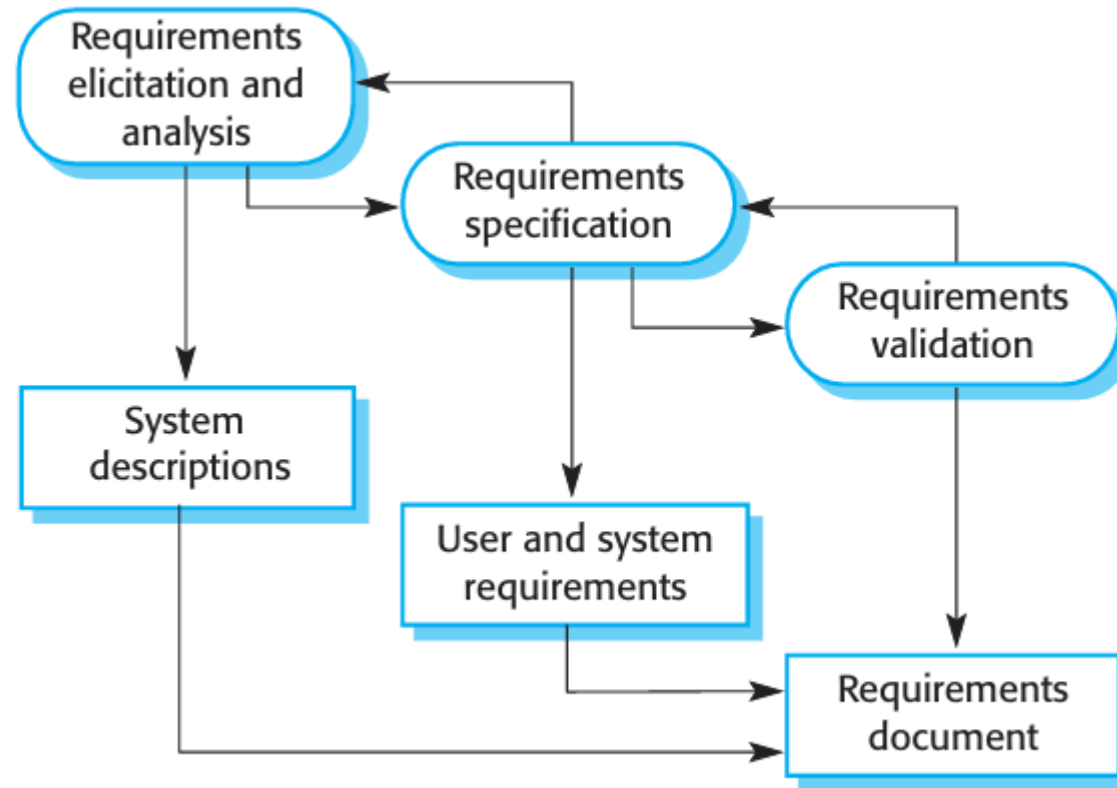


Figure 2.4 The requirements engineering process

Software Specification

There are three main activities in the requirements engineering process:

- **Requirements elicitation and analysis** : This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.
- **Requirements specification** : Requirements specification is the activity of translating the information gathered during requirements analysis into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a more detailed description of the functionality to be provided

Software Specification

There are three main activities in the requirements engineering process:

- **Requirements validation** :This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.
- Requirements analysis continues during definition and specification, and new requirements come to light throughout the process. Therefore, the activities of analysis, definition, and specification are interleaved.
- In agile methods, requirements specification is not a separate activity but is seen as part of system development. Requirements are informally specified for each increment of the system just before that increment is developed. Requirements are specified according to user priorities. The elicitation of requirements comes from users who are part of or work closely with the development team.

Software design and implementation

- The implementation stage of software development is the process of developing an executable system for delivery to the customer.
- Sometimes this involves separate activities of software design and programming.
- However, if an agile approach to development is used, design and implementation are interleaved, with no formal design documents produced during the process.
- Of course, the software is still designed, but the design is recorded informally on whiteboards and programmer's notebooks.
- A software design is a description of the structure of the software to be implemented, the data models and structures used by the system, the interfaces between system components and, sometimes, the algorithms used.
- Designers do not arrive at a finished design immediately but develop the design in stages. They add detail as they develop their design, with constant backtracking to modify earlier designs.

Software design and implementation

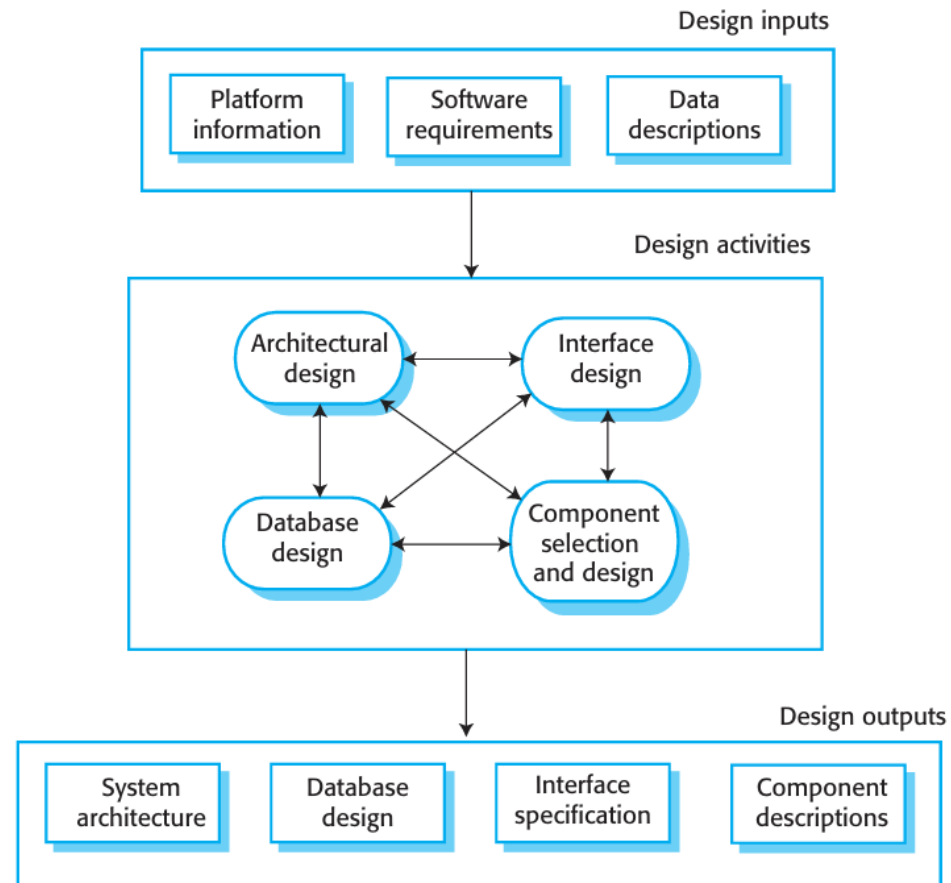


Figure 2.5 A general model of the design process

Software design and implementation

- The above figure is an abstract model of the design process showing the inputs to the design process, process activities, and the process outputs.
- The design process activities are both interleaved and interdependent. New information about the design is constantly being generated, and this affects previous design decisions. Design rework is therefore inevitable.
- Most software interfaces with other software systems. These other systems include the operating system, database, middleware, and other application systems. These make up the “software platform,” the environment in which the software will execute.

Software design and implementation

- Information about this platform is an essential input to the design process, as designers must decide how best to integrate it with its environment.
- If the system is to process existing data, then the description of that data may be included in the platform specification. Otherwise, the data description must be an input to the design process so that the system data organization can be defined.
- The activities in the design process vary, depending on the type of system being developed. For example, real-time systems require an additional stage of timing design but may not include a database, so there is no database design involved.

Software design and implementation

Four activities that may be part of the design process for information systems:

- **Architectural design**, where you identify the overall structure of the system, the principal components (sometimes called subsystems or modules), their relationships, and how they are distributed.
- **Database design**, where you design the system data structures and how these are to be represented in a database. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

Software design and implementation

- **Interface design**, where you define the interfaces between system components. This interface specification must be unambiguous. With a precise interface, a component may be used by other components without them having to know how it is implemented. Once interface specifications are agreed, the components can be separately designed and developed.
- **Component selection and design**, where you search for reusable components and, if no suitable components are available, design new software components. The design at this stage may be a simple component description with the implementation details left to the programmer. Alternatively, it may be a list of changes to be made to a reusable component or a detailed design model expressed in the UML. The design model may then be used to automatically generate an implementation

Software design and implementation

- For critical systems, the outputs of the design process are detailed design documents setting out precise and accurate descriptions of the system.
- The development of a program to implement a system follows naturally from system design.
- Although some classes of program, such as safety-critical systems, are usually designed in detail before any implementation begins, it is more common for design and program development to be interleaved.
- Software development tools may be used to generate a skeleton program from a design. This includes code to define and implement interfaces, and, in many cases, the developer need only add details of the operation of each program component.

Software design and implementation

- Programming is an individual activity, and there is no general process that is usually followed.
- Some programmers start with components that they understand, develop these, and then move on to less understood components.
- Others take the opposite approach, leaving familiar components till last because they know how to develop them.
- Some developers like to define data early in the process and then use this to drive the program development; others leave data unspecified for as long as possible.
- Normally, programmers carry out some testing of the code they have developed. This often reveals program defects (bugs) that must be removed from the program.

Software design and implementation

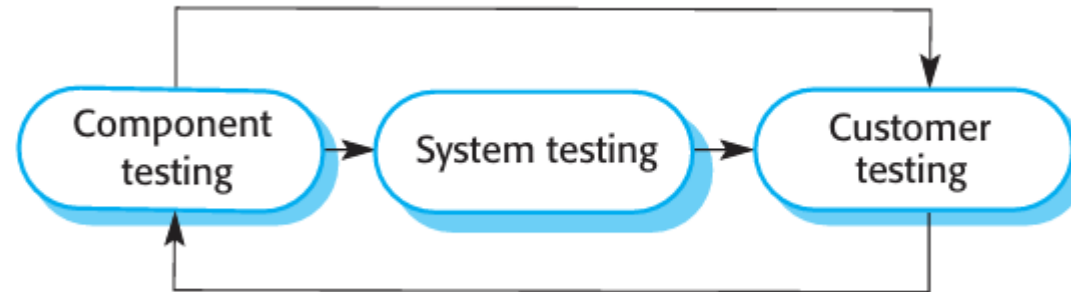
- Finding and fixing program defects is called debugging. Defect testing and debugging are different processes.
- ***Testing establishes the existence of defects. Debugging is concerned with locating and correcting these defects.***
- When you are debugging, you have to generate hypotheses about the observable behavior of the program and then test these hypotheses in the hope of finding the fault that caused the output anomaly.
- Testing the hypotheses may involve **tracing the program code manually**. It may require new test cases to localize the problem. Interactive debugging tools, which show the intermediate values of program variables and a trace of the statements executed, are usually used to support the debugging process

Software Validation

- Software validation or, more generally, verification and validation (V & V) is intended to show that a system both conforms to its specification and meets the expectations of the system customer.
- Program testing, where the system is executed using simulated test data, is the principal validation technique.
- Validation may also involve checking processes, such as inspections and reviews, at each stage of the software process from user requirements definition to program development. However, most V & V time and effort is spent on program testing.
- Except for small programs, systems should not be tested as a single, monolithic unit.

Software Validation

Figure 2.6 Stages of testing



Software Validation

- Figure shows a three-stage testing process in which system components are individually tested, then the integrated system is tested.
- For custom software, customer testing involves testing the system with real customer data.
- For products that are sold as applications, customer testing is sometimes called beta testing where selected users try out and comment on the software.

Software Validation

The stages in the testing process are:

- **Component testing** : The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes or may be coherent groupings of these entities. Test automation tools, such as JUnit for Java, that can rerun tests when new versions of the component are created, are commonly used (Koskela 2013).

Software Validation

The stages in the testing process are:

- **System testing** : System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties. For large systems, this may be a multistage process where components are integrated to form subsystems that are individually tested before these subsystems are integrated to form the final system.

Software Validation

The stages in the testing process are:

- **Customer testing** : This is the final stage in the testing process before the system is accepted for operational use. The system is tested by the system customer (or potential customer) rather than with simulated test data. For custom-built software, customer testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data. Customer testing may also reveal requirements problems where the system's facilities do not really meet the users' needs or the system performance is unacceptable. For products, customer testing shows how well the software product meets the customer's needs.

Software Validation

The stages in the testing process are:

- **Customer testing** : This is the final stage in the testing process before the system is accepted for operational use. The system is tested by the system customer (or potential customer) rather than with simulated test data. For custom-built software, customer testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data. Customer testing may also reveal requirements problems where the system's facilities do not really meet the users' needs or the system performance is unacceptable. For products, customer testing shows how well the software product meets the customer's needs.

Software Validation

- Ideally, component defects are discovered early in the testing process, and interface problems are found when the system is integrated.
- However, as defects are discovered, the program must be debugged, and this may require other stages in the testing process to be repeated.
- Errors in program components, say, may come to light during system testing. The process is therefore an iterative one with information being fed back from later stages to earlier parts of the process.
- Normally, component testing is simply part of the normal development process. Programmers make up their own test data and incrementally test the code as it is developed. The programmer knows the component and is therefore the best person to generate test cases.

Software Validation

- If an incremental approach to development is used, each increment should be tested as it is developed, with these tests based on the requirements for that increment.
- In test-driven development, which is a normal part of agile processes, tests are developed along with the requirements before development starts.
- This helps the testers and developers to understand the requirements and ensures that there are no delays as test cases are created.

Software Validation

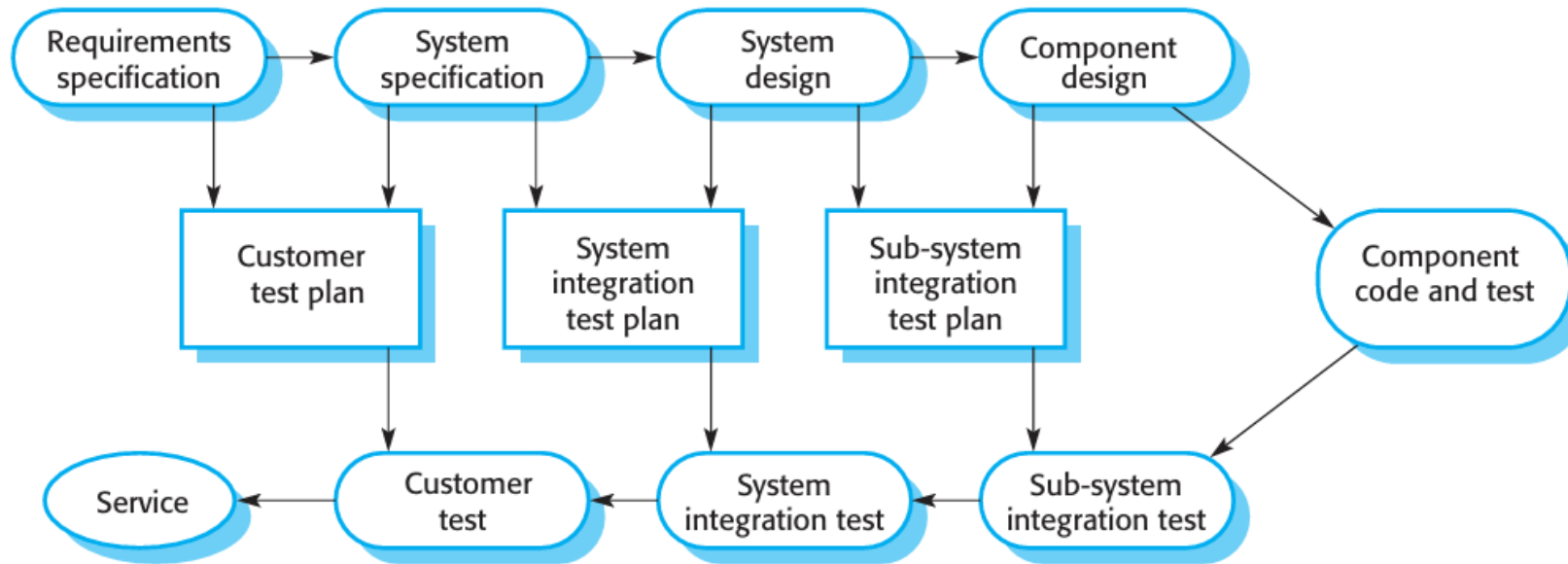


Figure 2.7 Testing phases in a plan-driven software process

Software Validation

- Figure 2.7 illustrates how test plans are the link between testing and development activities. This is sometimes called the V-model of development (turn it on its side to see the V). **The V-model shows the software validation activities that correspond to each stage of the waterfall process model.**
- **When a system is to be marketed as a software product, a testing process called beta testing is often used. Beta testing involves delivering a system to a number of potential customers who agree to use that system.** They report problems to the system developers.
- This exposes the product to real use and detects errors that may not have been anticipated by the product developers. After this feedback, the software product may be modified and released for further beta testing or general sale.

Software Evolution

- The flexibility of software is one of the main reasons why more and more software is being incorporated into large, complex systems.
- Once a decision has been made to manufacture hardware, it is very expensive to make changes to the hardware design.
- However, changes can be made to software at any time during or after the system development. Even extensive changes are still much cheaper than corresponding changes to system hardware.
- Historically, there has always been a split between the process of software development and the process of software evolution (software maintenance). People think of software development as a creative activity in which a software system is developed from an initial concept through to a working system.

Software Evolution

- However, they sometimes think of software maintenance as dull and uninteresting. They think that software maintenance is less interesting and challenging than original software development.
- This distinction between development and maintenance is increasingly irrelevant.
- Very few software systems are completely new systems, and it makes much more sense to see development and maintenance as a continuum.
- Rather than two separate processes, it is more realistic to think of software engineering as an evolutionary process where software is continually changed over its lifetime in response to changing requirements and customer needs.

Software Evolution

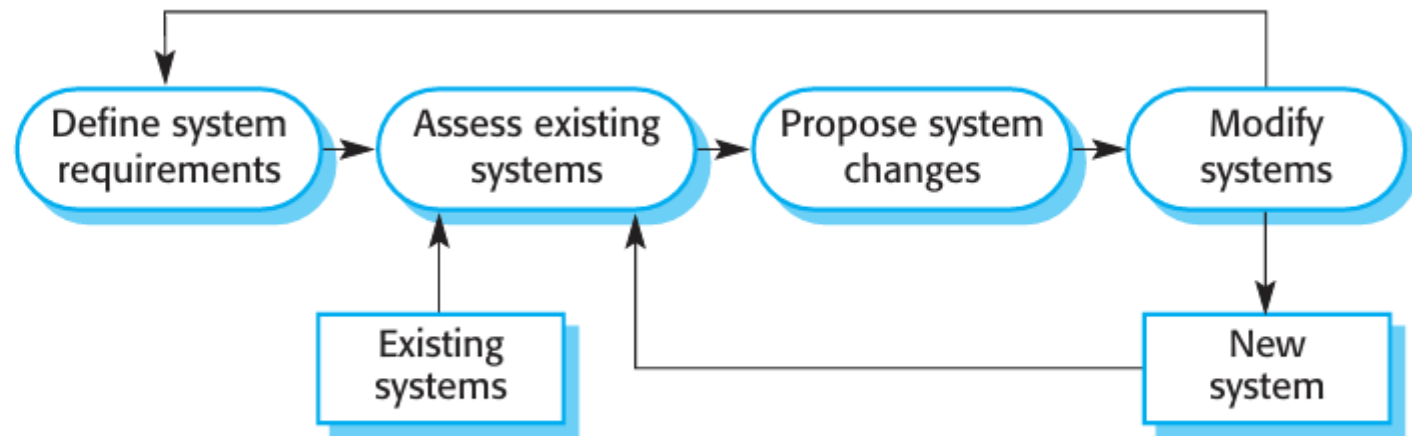


Figure 2.8 Software system evolution

Coping with Change

- Change is inevitable in all large software projects. The system requirements change as businesses respond to external pressures, competition, and changed management priorities.
- As new technologies become available, new approaches to design and implementation become possible.
- Therefore whatever software process model is used, it is essential that it can accommodate changes to the software being developed.
- Change adds to the costs of software development because it usually means that work that has been completed has to be redone. This is called rework.
- For example, if the relationships between the requirements in a system have been analyzed and new requirements are then identified, some or all of the requirements analysis has to be repeated.

Coping with Change

- It may then be necessary to redesign the system to deliver the new requirements, change any programs that have been developed, and retest the system.
- **Two related approaches may be used to reduce the costs of rework:**
 - **Change anticipation**, where the software process includes activities that can anticipate or predict possible changes before significant rework is required. For example, a prototype system may be developed to show some key features of the system to customers. They can experiment with the prototype and refine their requirements before committing to high software production costs.
 - **Change tolerance**, where the process and software are designed so that changes can be easily made to the system. This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have to be altered to incorporate the change.

Coping with Change

Two ways of coping with change and changing system requirements:

- **System prototyping**, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions. This is a method of change anticipation as it allows users to experiment with the system before delivery and so refine their requirements. The number of requirements change proposals made after delivery is therefore likely to be reduced.
- **Incremental delivery**, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the whole system and allows changes to be incorporated into later increments at relatively low cost.

Prototyping

- **A prototype is an early version of a software system that is used to demonstrate concepts, try out design options, and find out more about the problem and its possible solutions.**
- Rapid, iterative development of the prototype is essential so that costs are controlled and system stakeholders can experiment with the prototype early in the software process.
- A software prototype can be used in a software development process to help anticipate changes that may be required:
 - In the requirements engineering process, a prototype can help with the elicitation and validation of system requirements.
 - In the system design process, a prototype can be used to explore software solutions and in the development of a user interface for the system.

Prototyping

- System prototypes allow potential users to see how well the system supports their work.
- They may get new ideas for requirements and find areas of strength and weakness in the software. They may then propose new system requirements.
- Furthermore, as the prototype is developed, it may reveal errors and omissions in the system requirements. A feature described in a specification may seem to be clear and useful.
- However, when that function is combined with other functions, users often find that their initial view was incorrect or incomplete.
- The system specification can then be modified to reflect the changed understanding of the requirements.

Prototyping

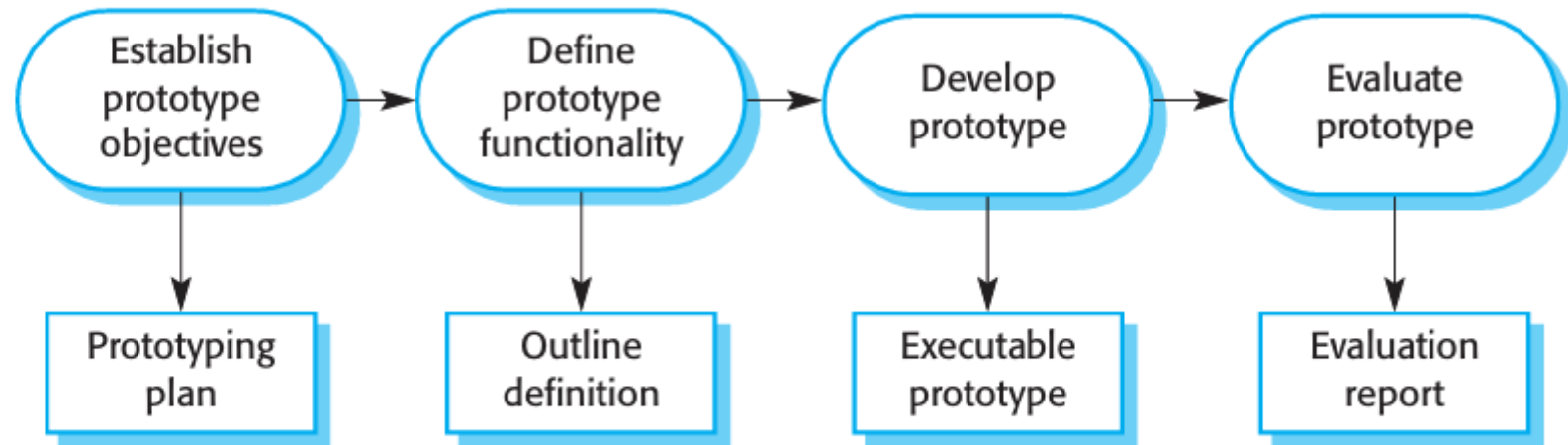


Figure 2.9 Prototype development

Prototyping

- A system prototype may be used while the system is being designed to carry out design experiments to check the feasibility of a proposed design.
- For example, a database design may be prototyped and tested to check that it supports efficient data access for the most common user queries. Rapid prototyping with end-user involvement is the only sensible way to develop user interfaces.
- Because of the dynamic nature of user interfaces, textual descriptions and diagrams are not good enough for expressing the user interface requirements and design.

Prototyping

- A process model for prototype development is shown in Figure 2.9. The objectives of prototyping should be made explicit from the start of the process.
- These may be to develop the user interface, to develop a system to validate functional system requirements, or to develop a system to demonstrate the application to managers.
- The same prototype usually cannot meet all objectives. If the objectives are left unstated, management or end-users may misunderstand the function of the prototype.
- Consequently, they may not get the benefits that they expected from the prototype development.

Prototyping

- The next stage in the process is to decide what to put into and, perhaps more importantly, what to leave out of the prototype system.
- To reduce prototyping costs and accelerate the delivery schedule, you may leave some functionality out of the prototype.
- You may decide to relax non-functional requirements such as response time and memory utilization.
- Error handling and management may be ignored unless the objective of the prototype is to establish a user interface.
- Standards of reliability and program quality may be reduced.

Prototyping

- The **final stage of the process is prototype evaluation**. Provision must be made during this stage for user training, and the prototype objectives should be used to derive a plan for evaluation.
- Potential users need time to become comfortable with a new system and to settle into a normal pattern of usage. Once they are using the system normally, they then discover requirements errors and omissions.
- A general problem with prototyping is that users may not use the prototype in the same way as they use the final system. Prototype testers may not be typical of system users. There may not be enough time to train users during prototype evaluation.
- If the prototype is slow, the evaluators may adjust their way of working and avoid those system features that have slow response times. When provided with better response in the final system, they may use it in a different way.

Incremental Delivery

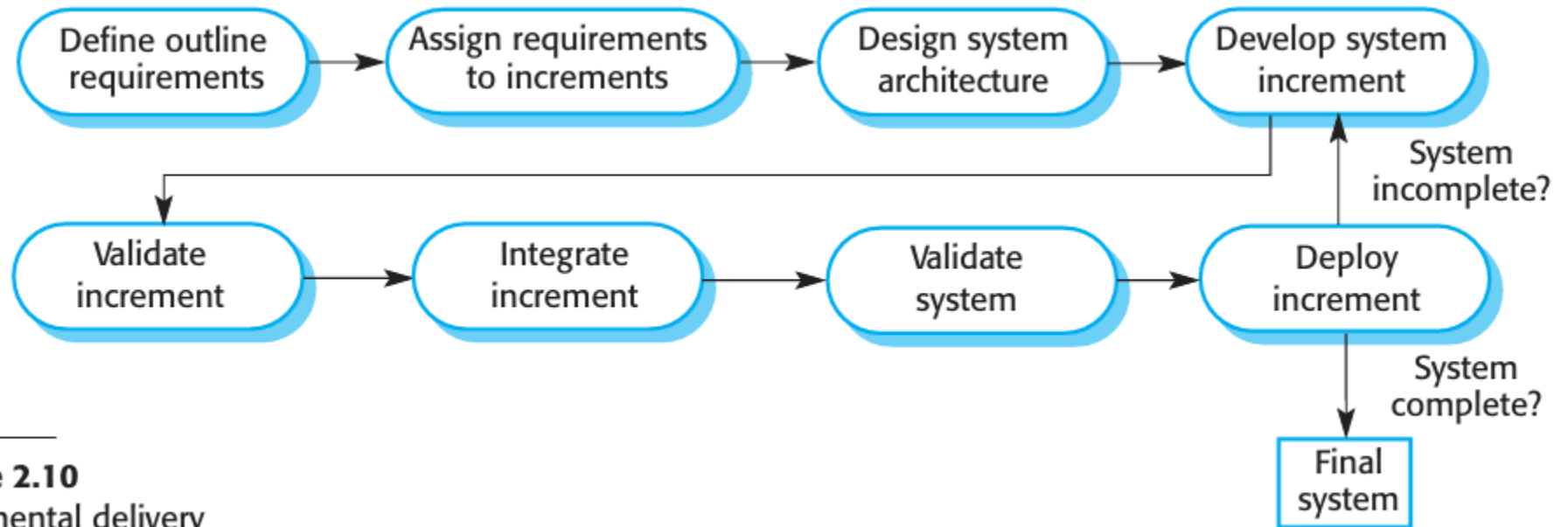


Figure 2.10
Incremental delivery

Incremental Delivery

- Incremental delivery is an approach to software development where some of the developed increments are delivered to the customer and deployed for use in their working environment.
- In an incremental delivery process, customers define which of the services are most important and which are least important to them.
- A number of delivery increments are then defined, with each increment providing a subset of the system functionality.
- The allocation of services to increments depends on the service priority, with the highest priority services implemented and delivered first.
- Once the system increments have been identified, the requirements for the services to be delivered in the first increment are defined in detail and that increment is developed.

Incremental Delivery

- During development, further requirements analysis for later increments can take place, but requirements changes for the current increment are not accepted.
- Once an increment is completed and delivered, it is installed in the customer's normal working environment. They can experiment with the system, and this helps them clarify their requirements for later system increments.
- As new increments are completed, they are integrated with existing increments so that system functionality improves with each delivered increment.

Incremental Delivery

Incremental delivery has a number of advantages:

- Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments. Unlike prototypes, these are part of the real system, so there is no relearning when the complete system is available.
- Customers do not have to wait until the entire system is delivered before they can gain value from it. The first increment satisfies their most critical requirements, so they can use the software immediately.
- The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes into the system.

Incremental Delivery

Incremental delivery has a number of advantages:

- As the highest priority services are delivered first and later increments then integrated, the most important system services receive the most testing. This means that customers are less likely to encounter software failures in the most important parts of the system.
- However, there are problems with incremental delivery. In practice, it only works in situations where a brand-new system is being introduced and the system evaluators are given time to experiment with the new system. Key problems with this approach are:
 - **Iterative delivery is problematic when the new system is intended to replace an existing system.** Users need all of the functionality of the old system and are usually unwilling to experiment with an incomplete new system. It is often impractical to use the old and the new systems alongside each other as they are likely to have different databases and user interfaces.

Incremental Delivery

- However, there are problems with incremental delivery. In practice, it only works in situations where a brand-new system is being introduced and the system evaluators are given time to experiment with the new system. Key problems with this approach are:
 - **Most systems require a set of basic facilities that are used by different parts of the system.** As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
 - **The essence of iterative processes is that the specification is developed in conjunction with the software.** However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract. In the incremental approach, there is no complete system specification until the final increment is specified. This requires a new form of contract, which large customers such as government agencies may find difficult to accommodate.

Incremental Delivery

- For some types of systems, incremental development and delivery is not the best approach.
- These are very large systems where development may involve teams working in different locations, some embedded systems where the software depends on hardware development, and some critical systems where all the requirements must be analyzed to check for interactions that may compromise the safety or security of the system.
- These large systems, of course, suffer from the same problems of uncertain and changing requirements. Therefore, to address these problems and get some of the benefits of incremental development, a system prototype may be developed and used as a platform for experiments with the system requirements and design.

Process Improvement

- Nowadays, there is a constant demand from industry for cheaper, better software, which has to be delivered to ever-tighter deadlines.
- Consequently, many software companies have turned to software process improvement as a way of enhancing the quality of their software, reducing costs, or accelerating their development processes.
- Process improvement means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

Process Improvement

Two quite different approaches to process improvement and change are used:

- **The process maturity approach, which has focused on improving process and project management and introducing good software engineering practice into an organization.** The level of process maturity reflects the extent to which good technical and management practice has been adopted in organizational software development processes. The primary goals of this approach are improved product quality and process predictability.
- **The agile approach, which has focused on iterative development and the reduction of overheads in the software process.** The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements. The improvement philosophy here is that the best processes are those with the lowest overheads and agile approaches can achieve this.

Process Improvement

- The process maturity approach is rooted in plan-driven development and usually requires increased “overhead,” in the sense that activities are introduced that are not directly relevant to program development.
- Agile approaches focus on the code being developed and deliberately minimize formality and documentation.

Process Improvement

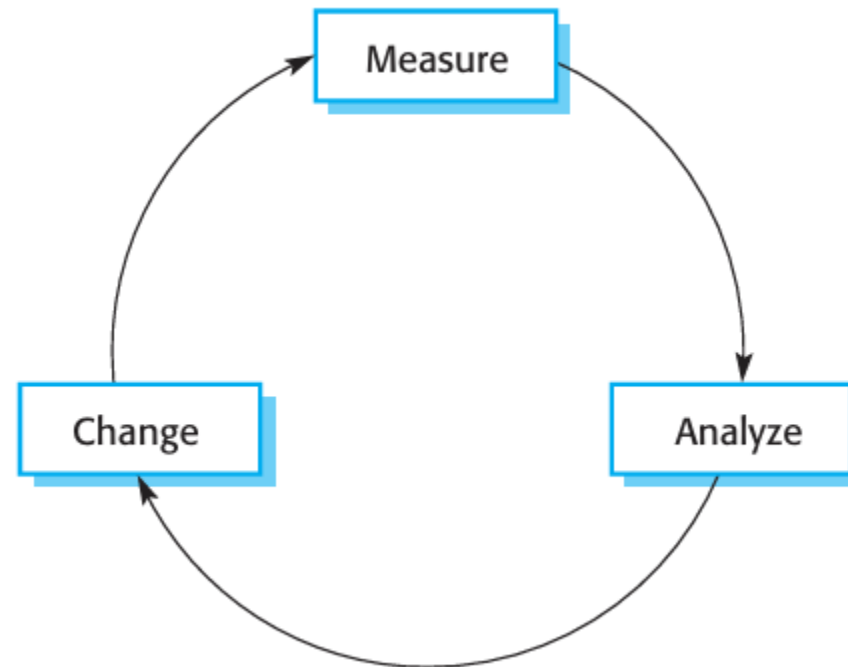


Figure 2.11 The process improvement cycle

Process Improvement

- **Process measurement** : You measure one or more attributes of the software process or product. These measurements form a baseline that helps you decide process improvements have been effective. As you introduce improvements, you re-measure the same attributes, which will hopefully have improved in some way.
- **Process analysis** : The current process is assessed, and process weaknesses and bottlenecks are identified. Process models (sometimes called process maps) that describe the process may be developed during this stage. The analysis may be focused by considering process characteristics such as rapidity and robustness.
- **Process change** : Process changes are proposed to address some of the identified process weaknesses. These are introduced, and the cycle resumes to collect data about the effectiveness of the changes.

Process Improvement

- Process improvement is a long-term activity, so each of the stages in the improvement process may last several months.
- It is also a continuous activity as, whatever new processes are introduced, the business environment will change and the new processes will themselves have to evolve to take these changes into account.
- The fundamental ideas in Humphrey's model are still the basis of software process maturity assessment. The levels in the process maturity model are given as below:

Process Improvement

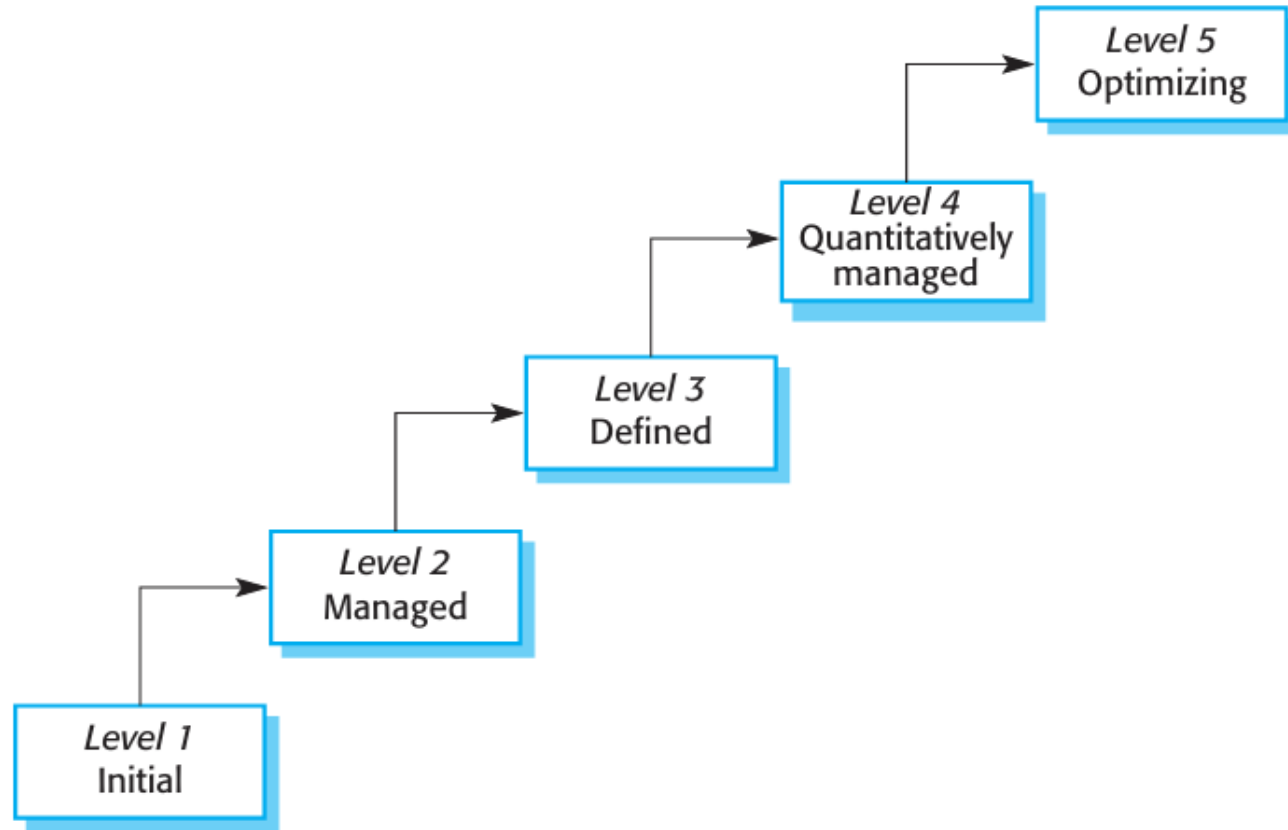


Figure 2.12 Capability maturity levels

Process Improvement

- **Initial** : The goals associated with the process area are satisfied, and for all processes the scope of the work to be performed is explicitly set out and communicated to the team members.
- **Managed** : At this level, the goals associated with the process area are met, and organizational policies are in place that define when each process should be used. There must be documented project plans that define the project goals. Resource management and process monitoring procedures must be in place across the institution.
- **Defined** : This level focuses on organizational standardization and deployment of processes. Each project has a managed process that is adapted to the project requirements from a defined set of organizational processes. Process assets and process measurements must be collected and used for future process improvements.

Process Improvement

- **Quantitatively managed** : At this level, there is an organizational responsibility to use statistical and other quantitative methods to control subprocesses. That is, collected process and product measurements must be used in process management.
- **Optimizing** : At this highest level, the organization must use the process and product measurements to drive process improvement. Trends must be analyzed and the processes adapted to changing business needs.
- The work on process maturity levels has had a major impact on the software industry. It focused attention on the software engineering processes and practices that were used and led to significant improvements in software engineering capability.

References

- Software Engineering, 10th Edition, Ian Sommerville, Pearson Education 2016
- Software Engineering: A Practitioner's Approach, 8th Edition, Roger S. Pressman and Bruce R. Maxim, McGraw-Hill Education 2015
- Beginning Software Engineering, Rod Stephens, John Wiley & Sons Inc 2015